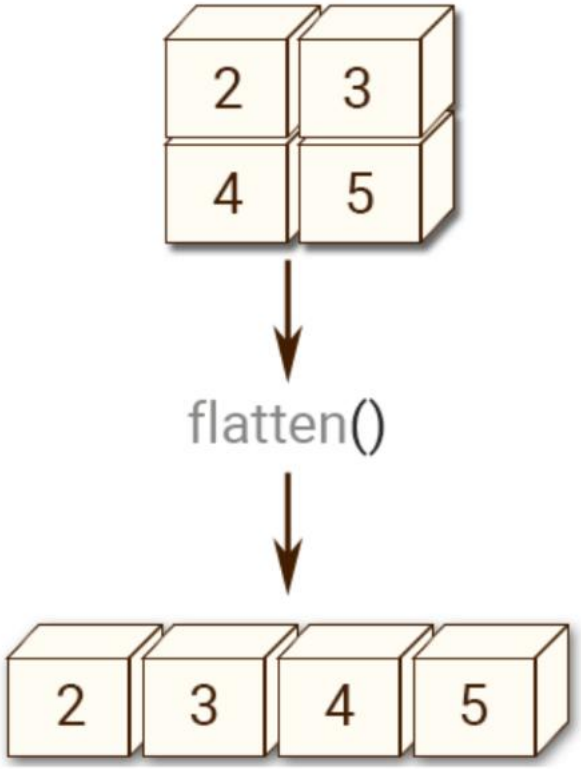
Computer Vision

DPS920

Session 17

ML in Images

ML in Images



Fire Detection

```python
import glob
import cv2
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import warnings
warnings.filterwarnings('ignore')
from joblib import dump

def load_data():
    data_list = []
    labels = []

    for i, address in enumerate(glob.glob('S18/fire_dataset\\*\\*')):
        img = cv2.imread(address)
        img = cv2.resize(img, (32,32))
        img = img/255
        img = img.flatten()

        data_list.append(img)
        label = address.split("\\")[-1].split(".")[0]
        labels.append(label)

        if i % 100 == 0:
            print(f"[INFO]: {i}/1000 processed")

    data_list = np.array(data_list)

    X_train, X_test, y_train, y_test = train_test_split(data_list, labels, test_size=0.2)

    return X_train, X_test, y_train, y_test

X_train, X_test, y_train, y_test = load_data()
clf = KNeighborsClassifier()
clf.fit(X_train, y_train)
predictions = clf.predict(X_test)
print(accuracy_score(y_test, predictions))
dump(clf, 'fire_detection.z')
```

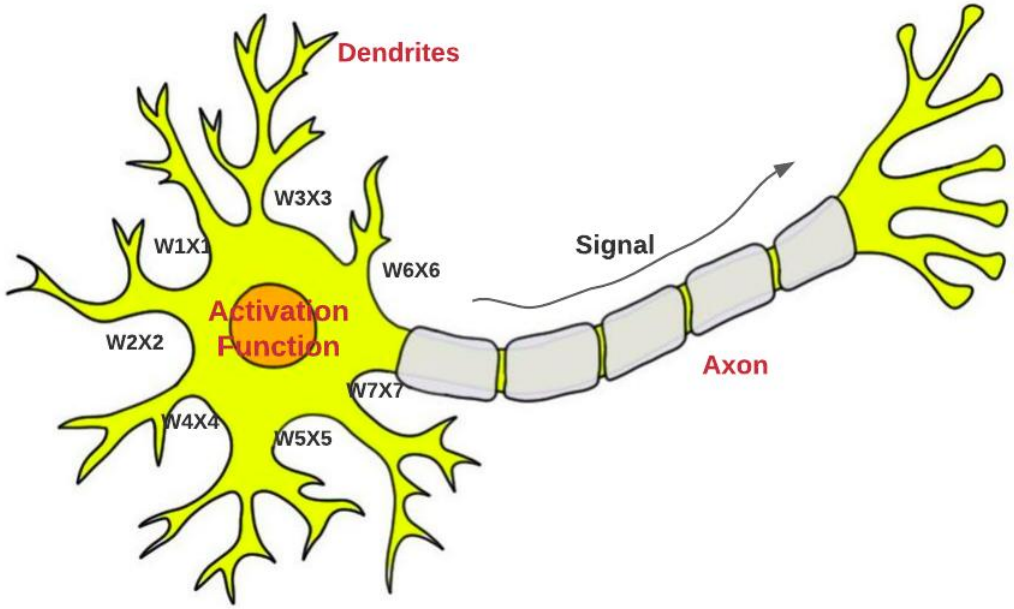
```python
import cv2
import numpy as np
import glob
from joblib import load

clf = load('S18/fire_detector.z')

for item in glob.glob("test_images\\*"):
    img = cv2.imread(item)
    r_img = cv2.resize(img, (32,32))
    r_img = img/255
    r_img = img.flatten()
    r_img = np.array([r_img])

    label = clf.predict(r_img)[0]

    cv2.putText(img, label, (10, 40), cv2.FONT_HERSHEY_SIMPLEX, 1.9, (0,255,0), 2)
    cv2.imshow()
    cv2.waitKey(0)

cv2.destroyAllWindows()
```

Types of Preprocessing in Images

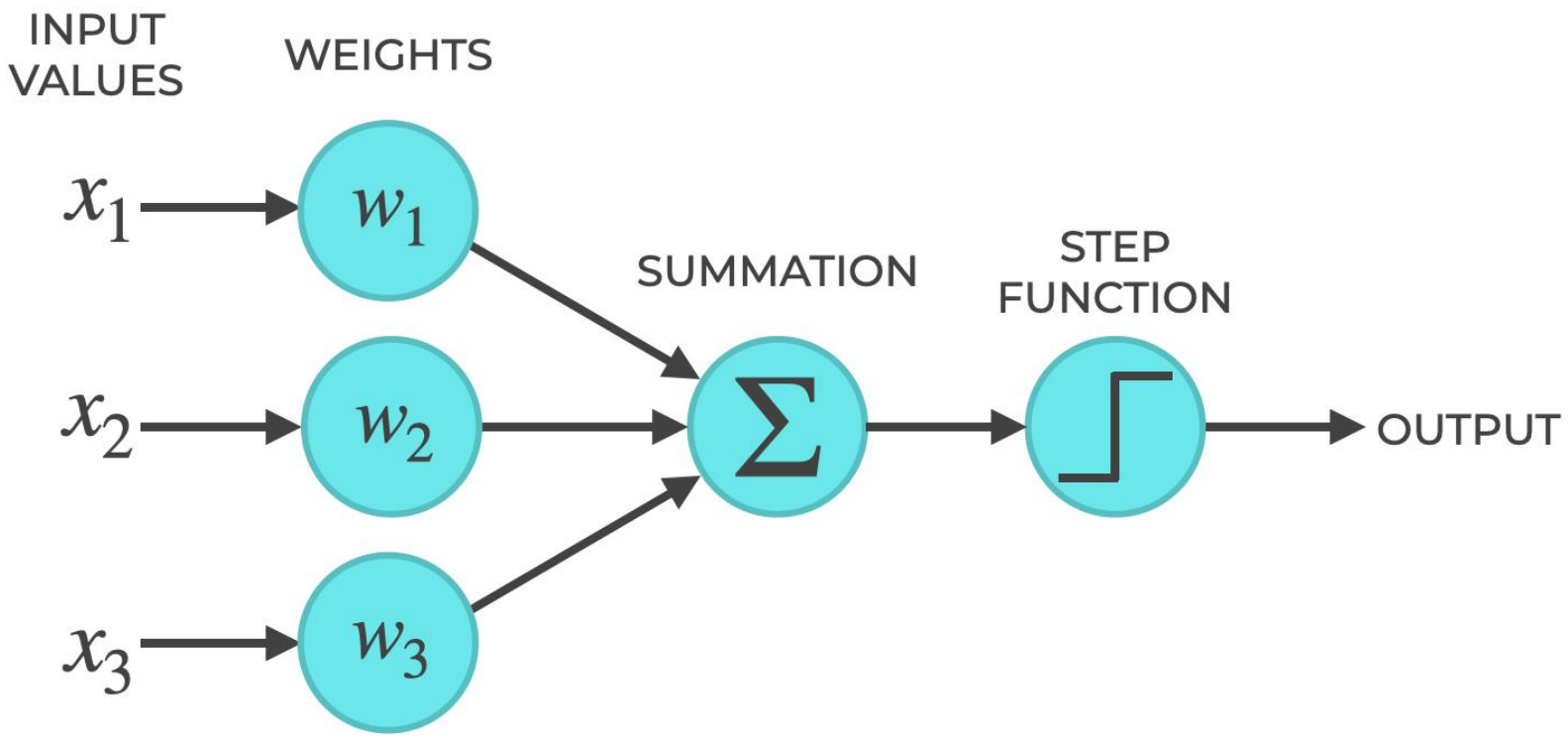
- Resize
- Normalization
- Flat

Brain

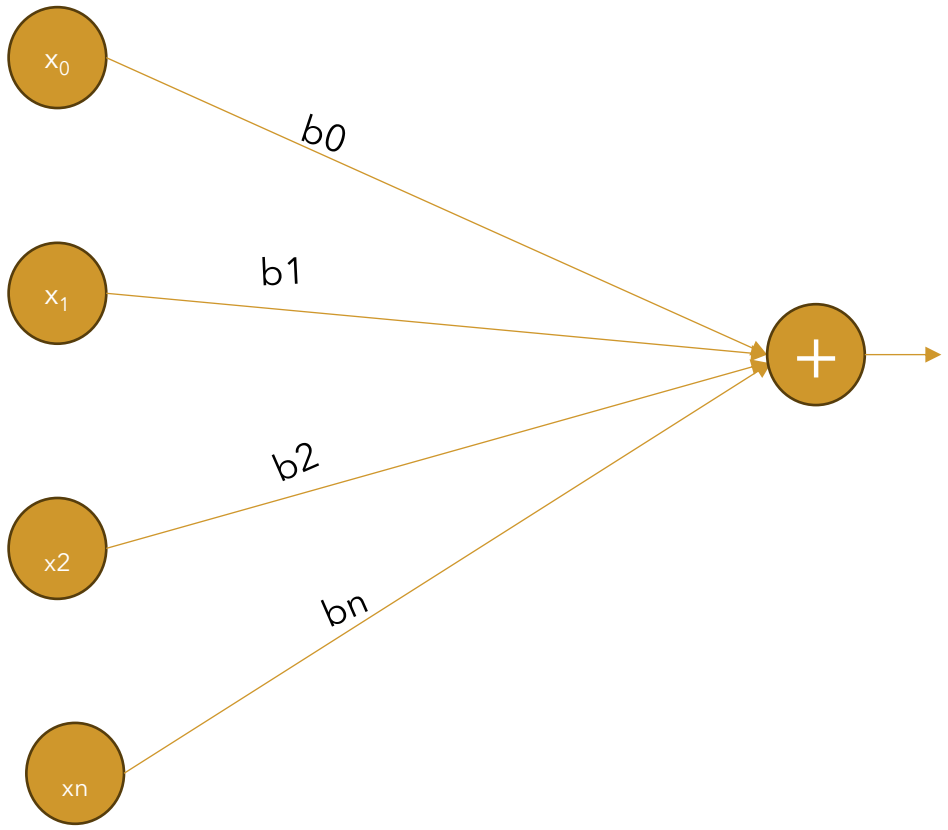
Perceptron


Dendrites
W3X3
W1X1
W6X6
Signal
Activation
Function
W2X2
Axon
W7X7
W4X4
W5X5

Perceptron Algorithm

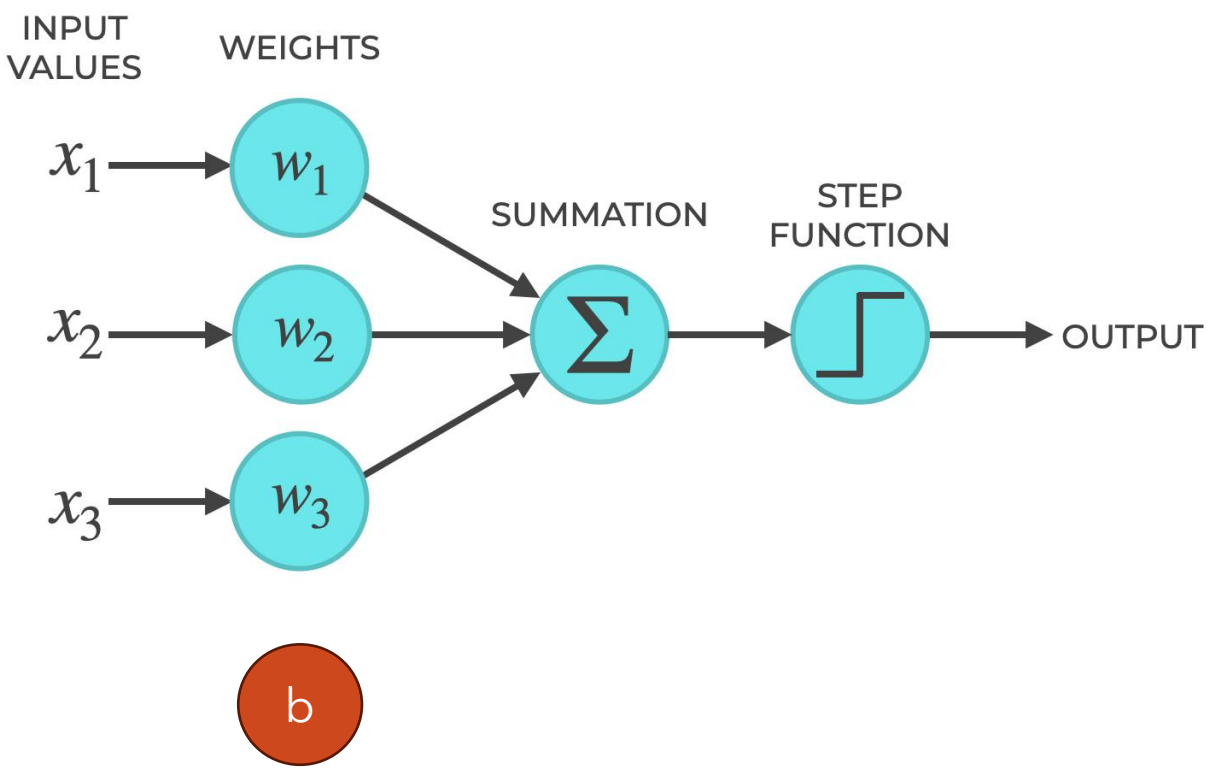


MLR



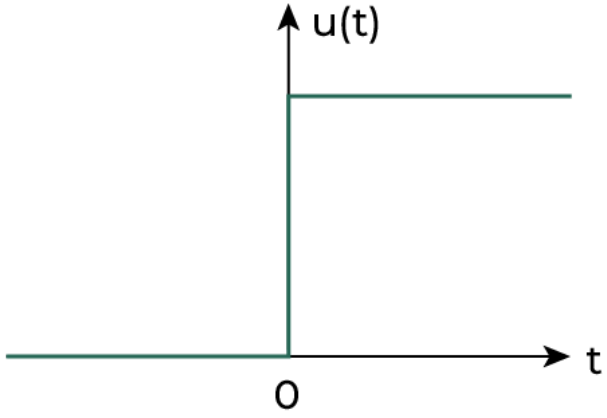
$$y = b_0 + b_1 * x_1 + b_2 * x_2 + \ldots + b_n * x_n$$

Perceptron Algorithm



$$\begin{cases} f(x_0w_0 + x_1w_1 + x_2w_2 + x_3w_3) \\ f(\sum w_i\, x_i) \\ f(\text{net}) \end{cases}$$

Step Function

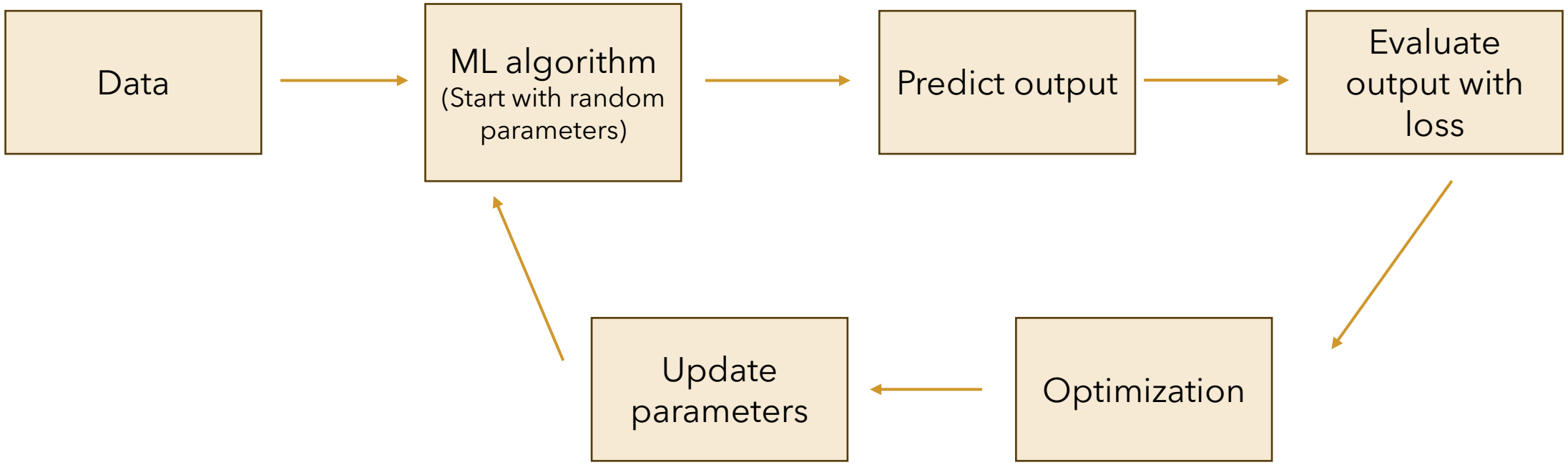


$$u(t) = \begin{cases} 1 & t > 0 \\ 0 & t > 0 \end{cases}$$

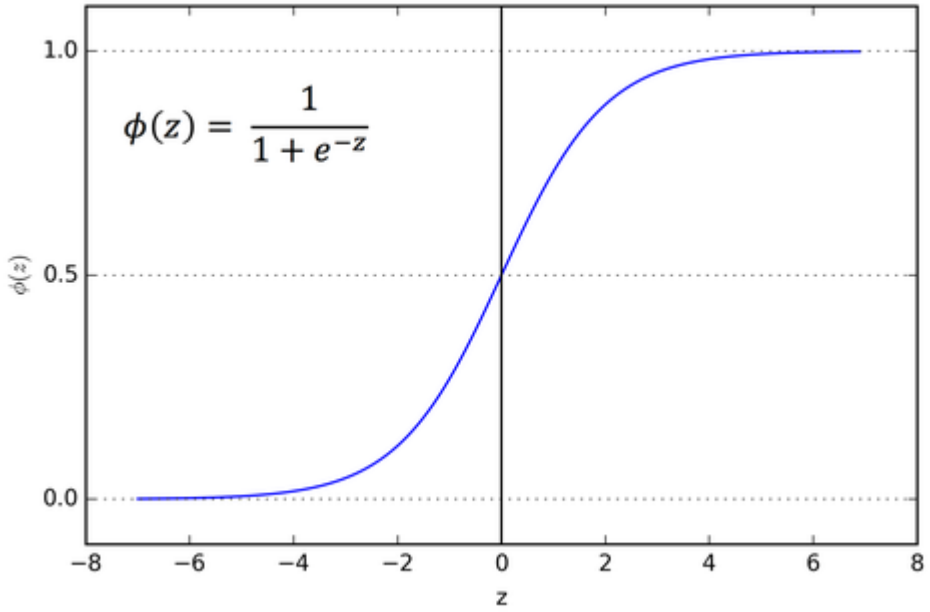
Pros: simple

Cons: no derivates

Framework


Data
ML algorithm
(Start with random parameters)
Predict output
Evaluate output with loss
Optimization
Update parameters

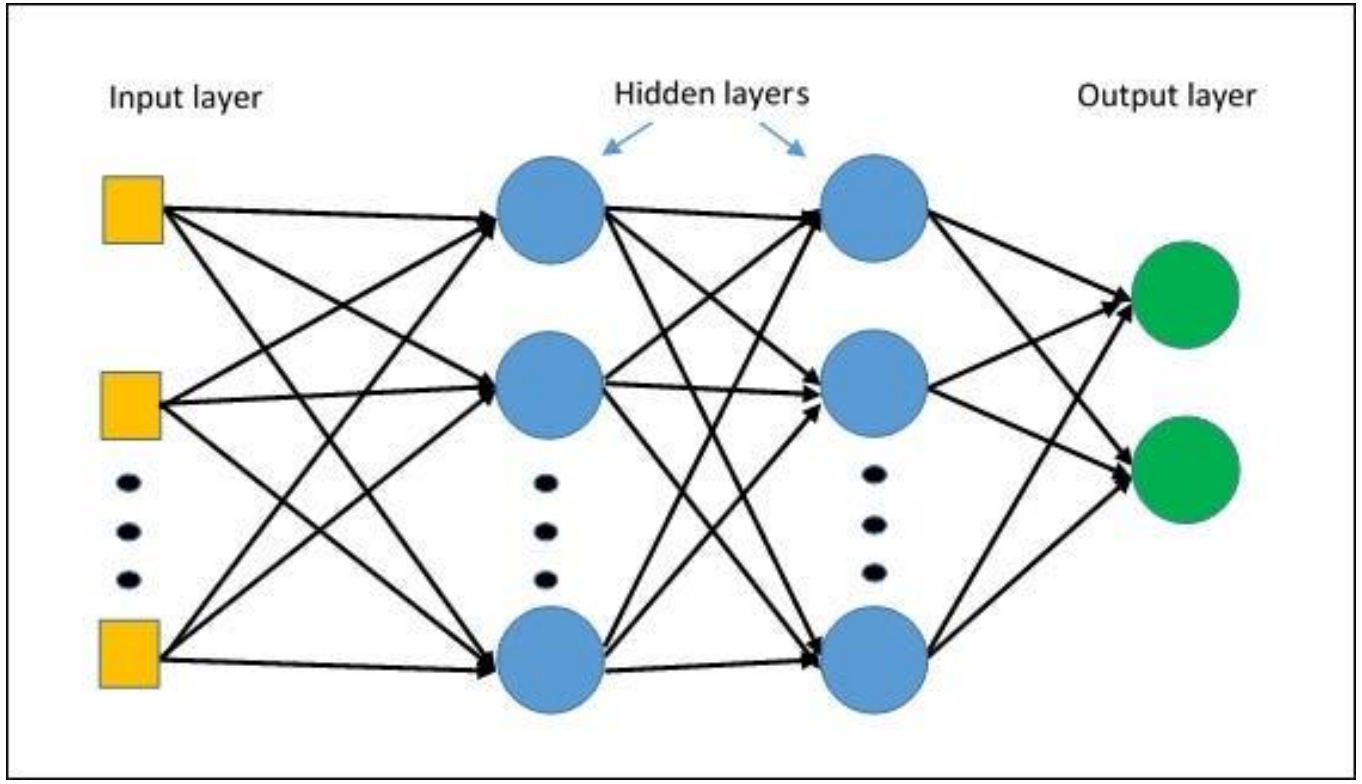
Sigmoid



Pros: derivative at all points

Cons: small derivatives at end points

Multi Layer Perceptron



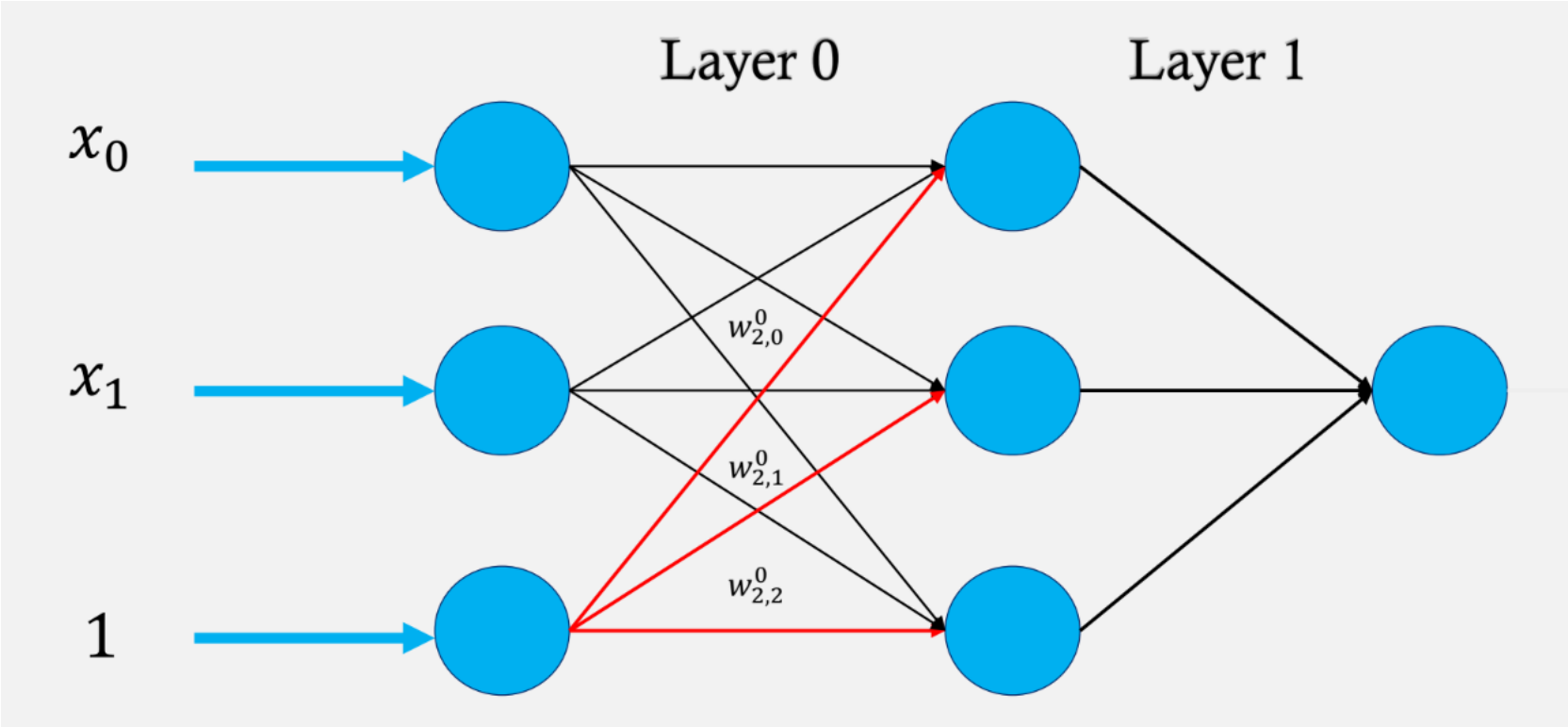
2 main steps

Forward Pass

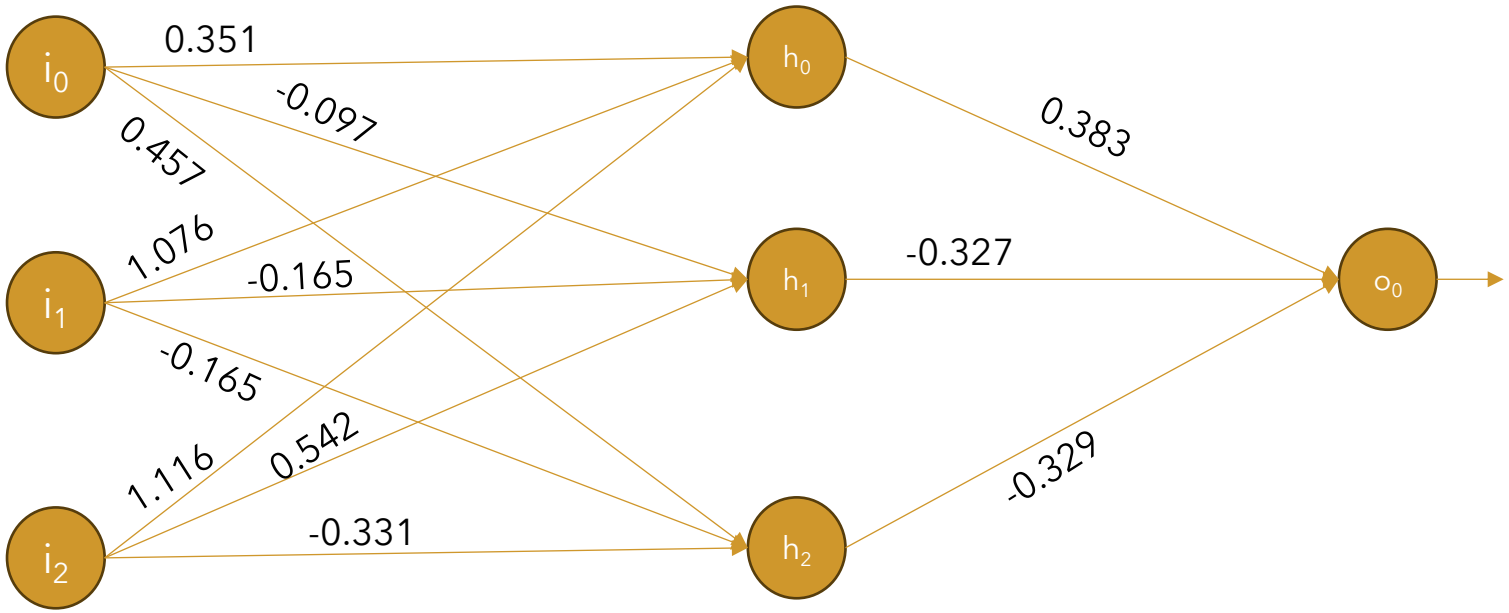
Backward Pass

Forward Pass

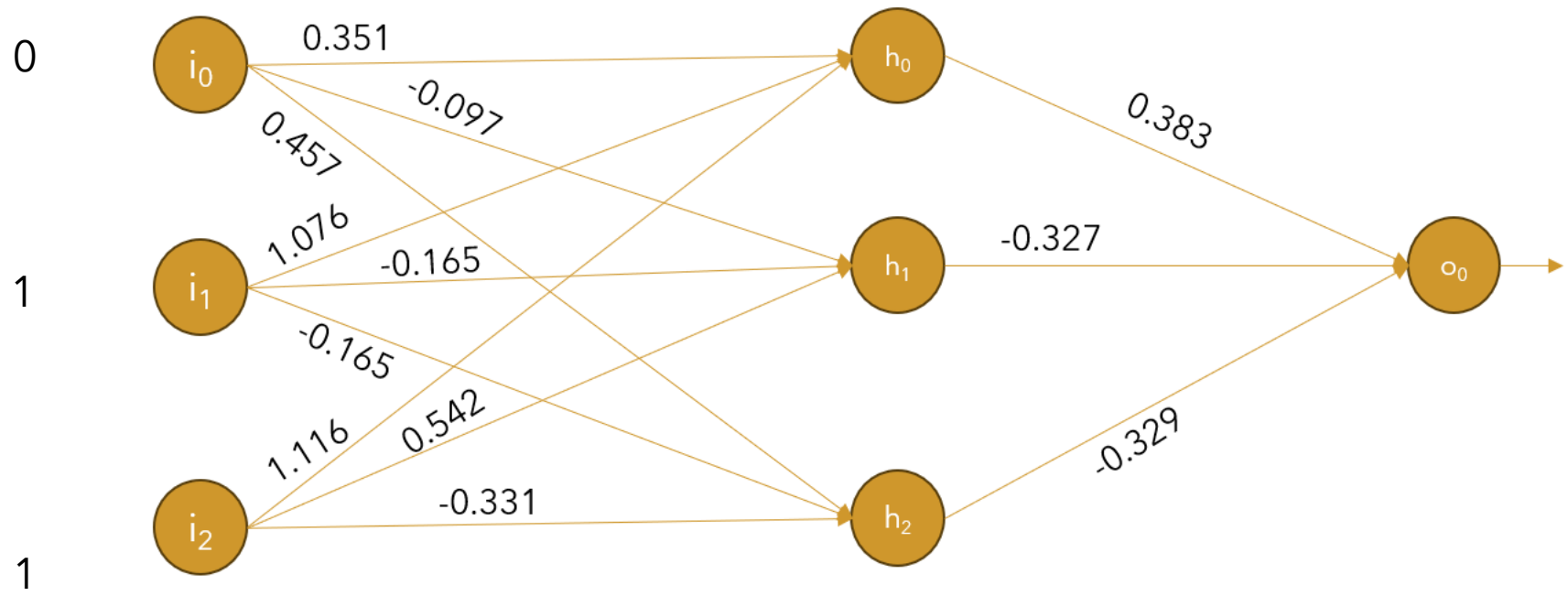
Forward Pass



Example

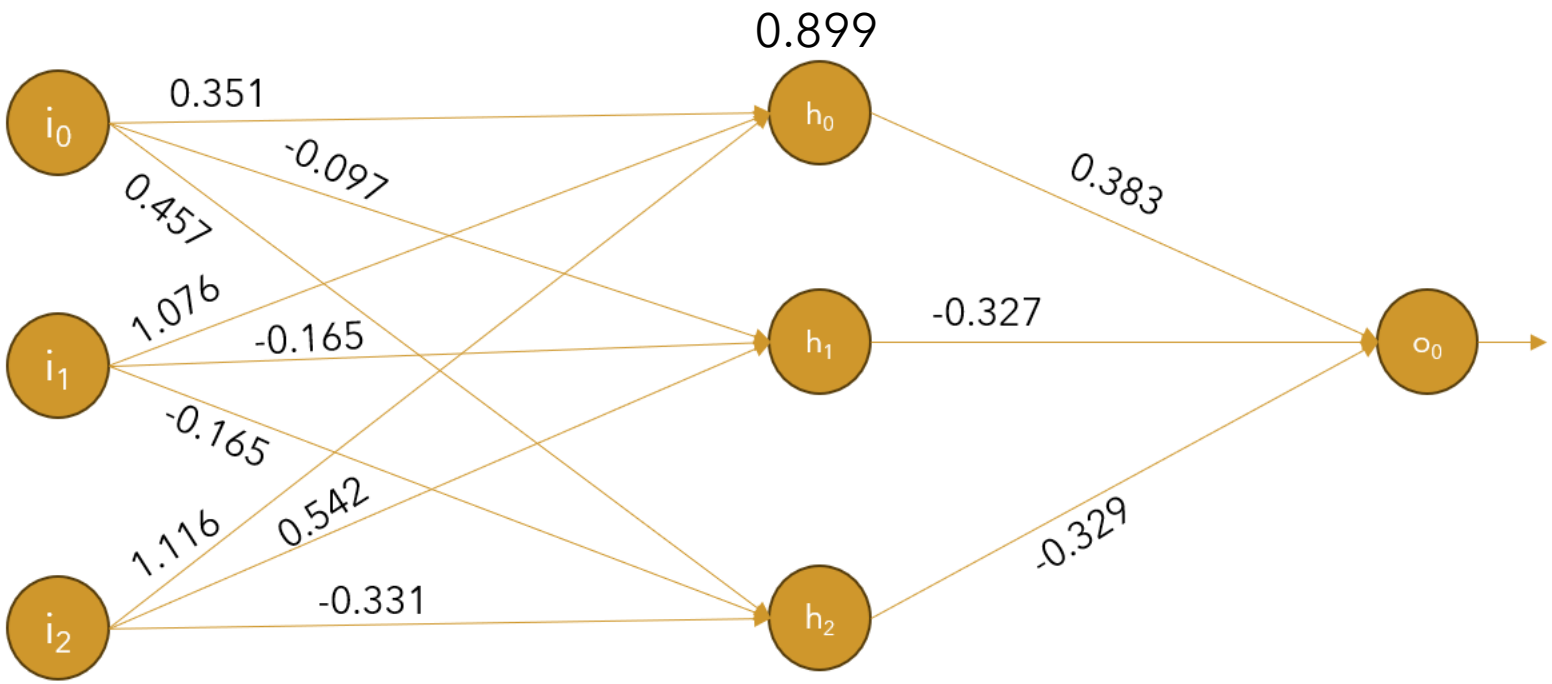

i_0
i_1
i_2
h_0
h_1
h_2
o_0
0.351
-0.097
0.457
1.076
-0.165
-0.165
1.116
0.542
-0.331
0.383
-0.327
-0.329

Example



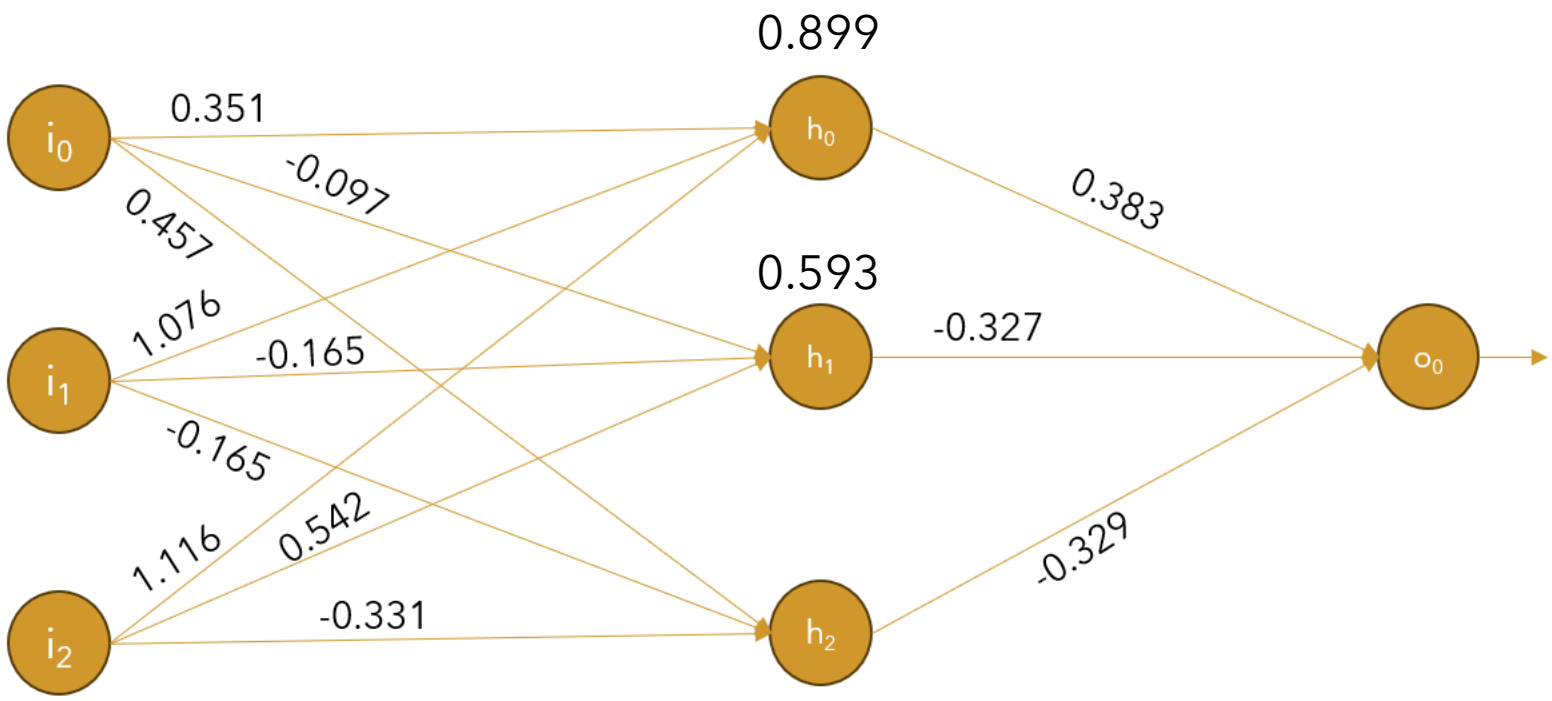
$$h_0 = 0(0.351) + 1(1.076) + 1(1.116) = 2.192$$

Example

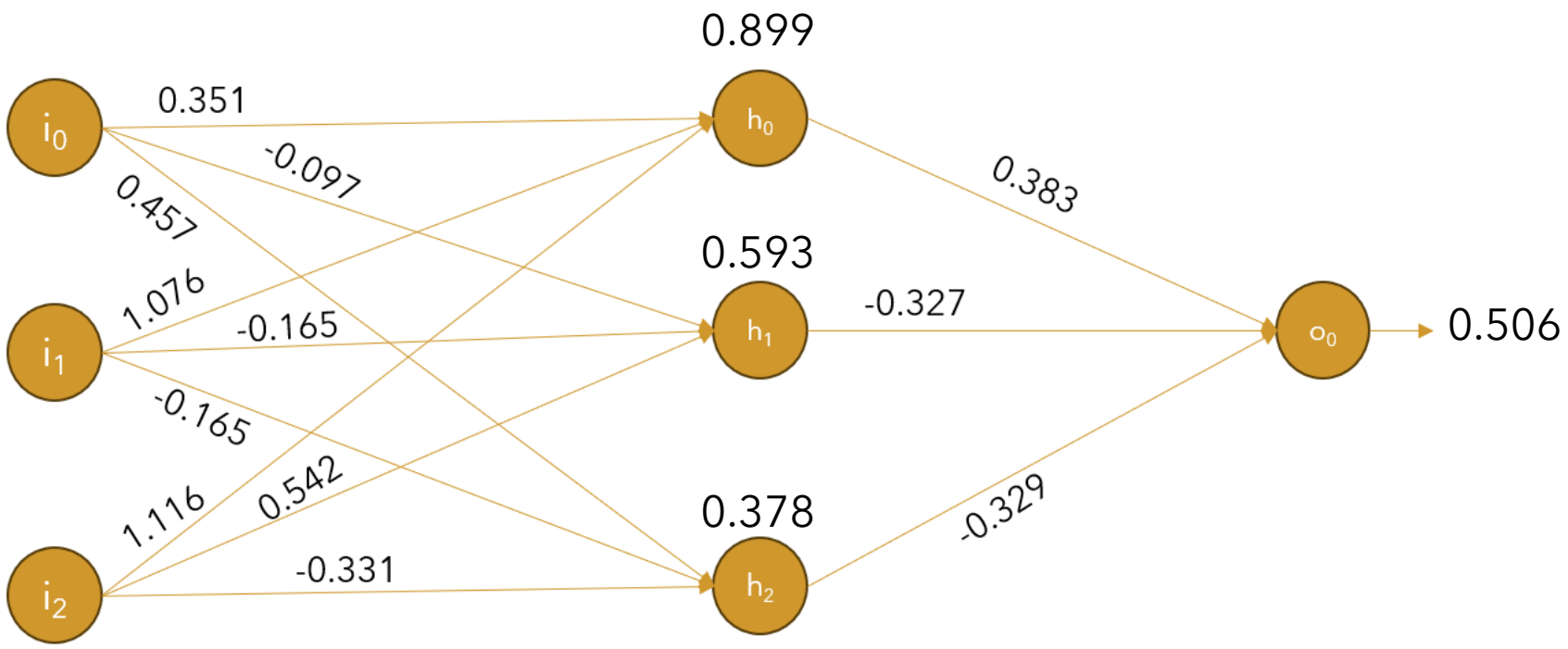


$$\frac{1}{1 + e^{-2.192}} = 0.899$$

Exmaple

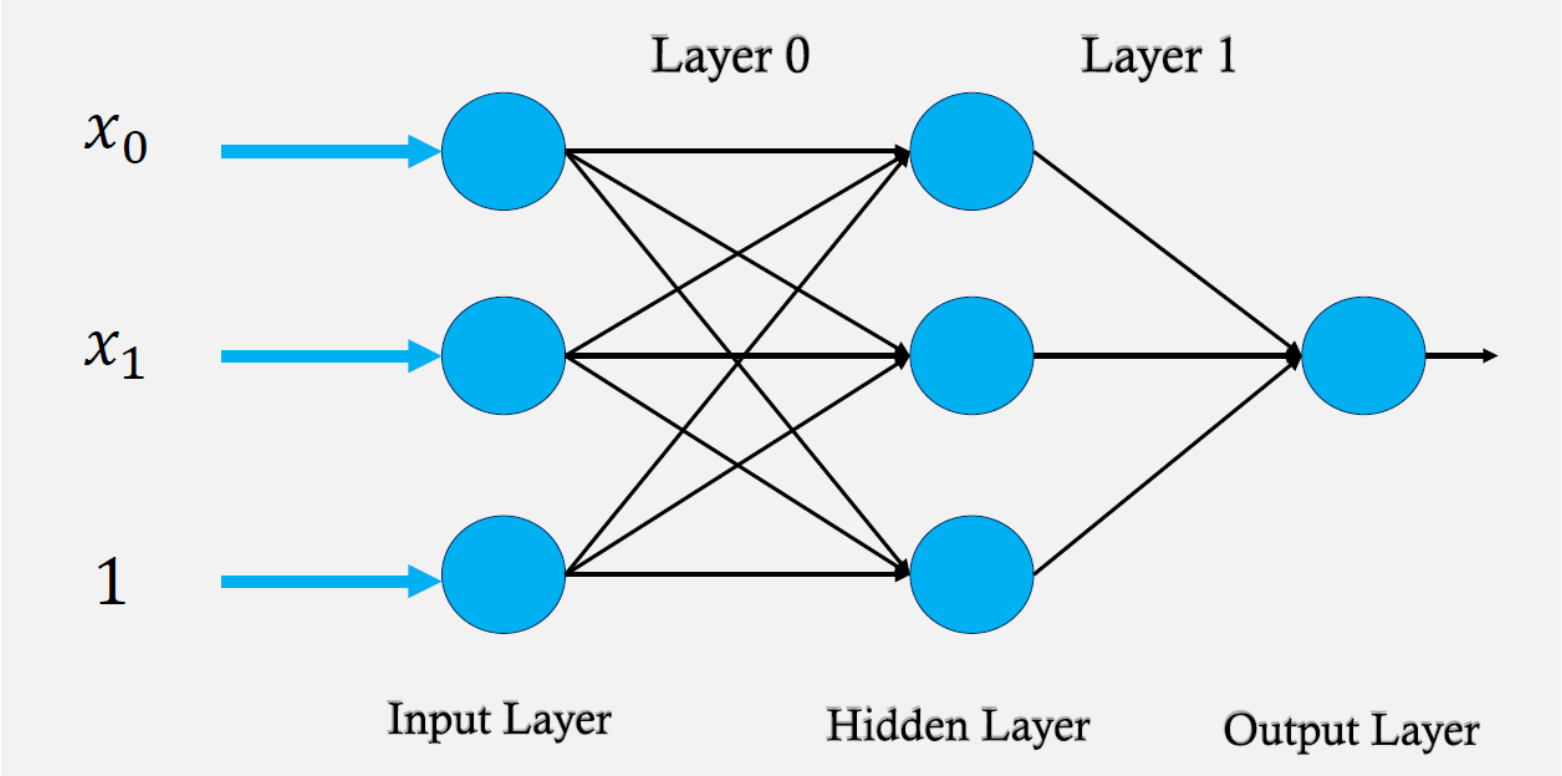

0.899
0.593
i_0
i_1
i_2
h_0
h_1
h_2
o_0
0.351
-0.097
0.457
1.076
-0.165
-0.165
1.116
0.542
-0.331
0.383
-0.327
-0.329

Exmaple


0.899
0.593
0.378
0.506
i_0
i_1
i_2
h_0
h_1
h_2
o_0
0.351
-0.097
0.457
1.076
-0.165
-0.165
1.116
0.542
-0.331
0.383
-0.327
-0.329

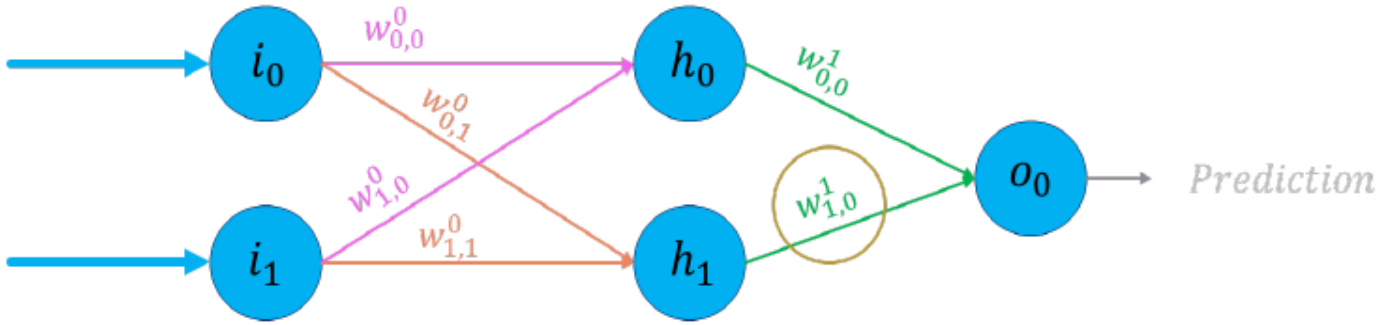
Backward Pass

Backpropagation

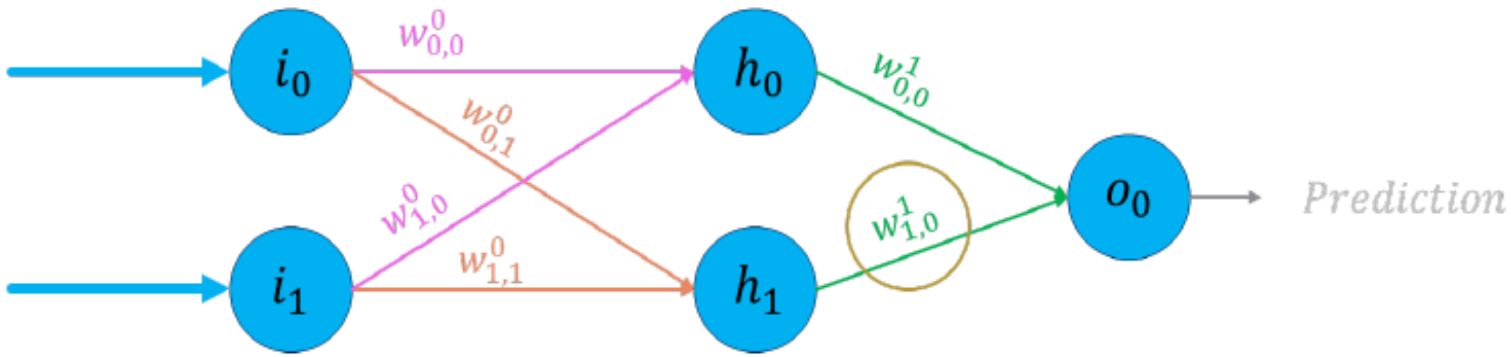


Simpler Backpropagation

For simplicity let's only consider summation and multiplications (no activation or bias)



Backpropagation

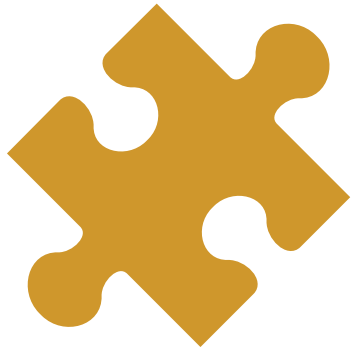


Prediction = $(x_0 w^0_{0,0} + x_0 w^0_{1,0})\, w^1_{0,0} + (x_0 w^0_{0,1} + x_0 w^0_{1,1})\, w^1_{1,0}$

$$\text{Loss} = \frac{(prediction - actual)^2}{2}$$

Goal

Final goal is to achieve to the best value for parameters

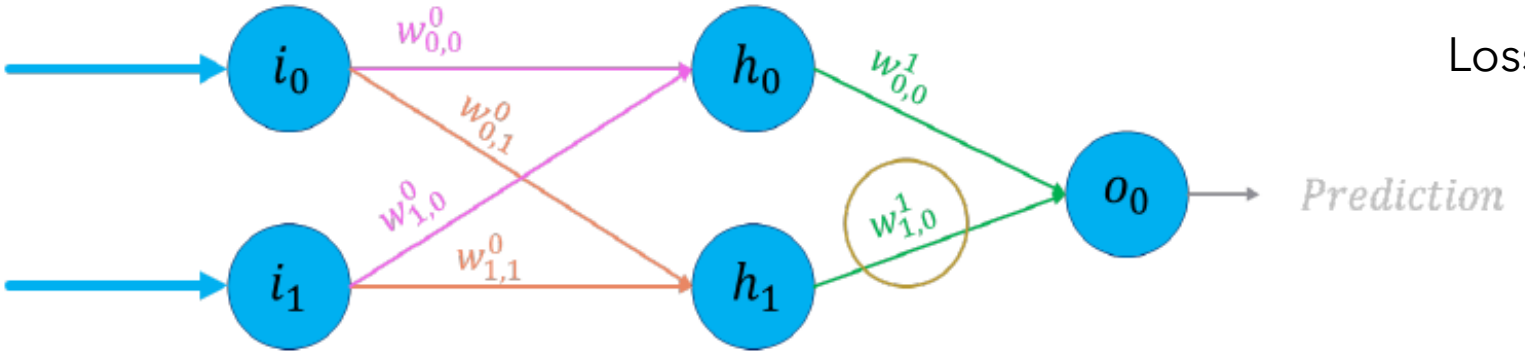


What are the parameters?

Backpropagation

Prediction = $(x_0 w^0_{0,0} + x_0 w^0_{1,0})\, w^1_{0,0} + (x_0 w^0_{0,1} + x_0 w^0_{1,1})\, w^1_{1,0}$

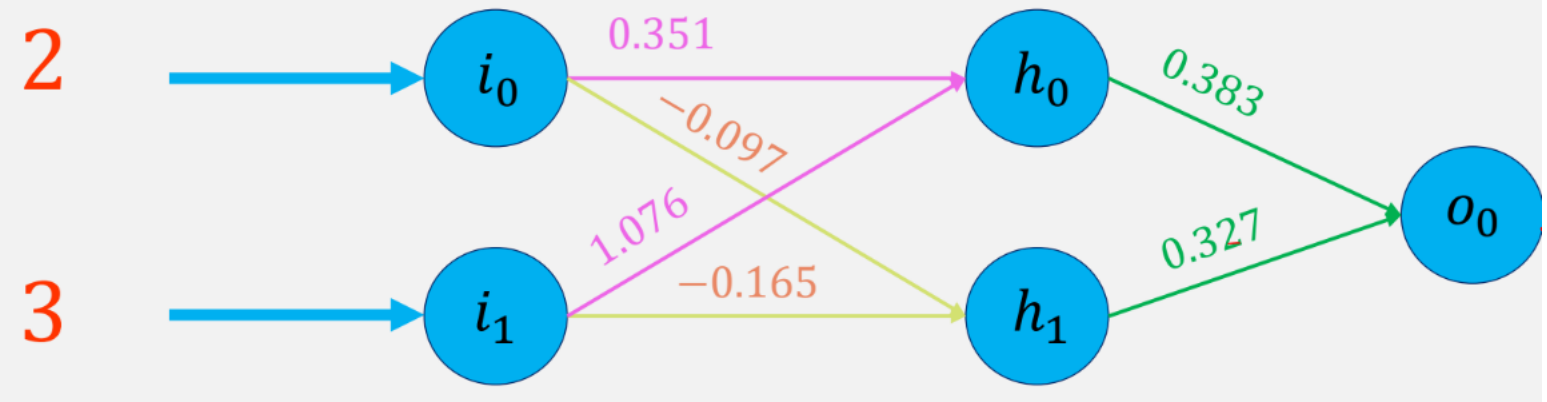
Loss = $\frac{(prediction - actual)^2}{2}$



$$\frac{\partial loss}{\partial w^1_{1,0}} = \frac{\partial loss}{\partial Prediction} \times \frac{\partial Prediction}{\partial w^1_{1,0}}$$

Example

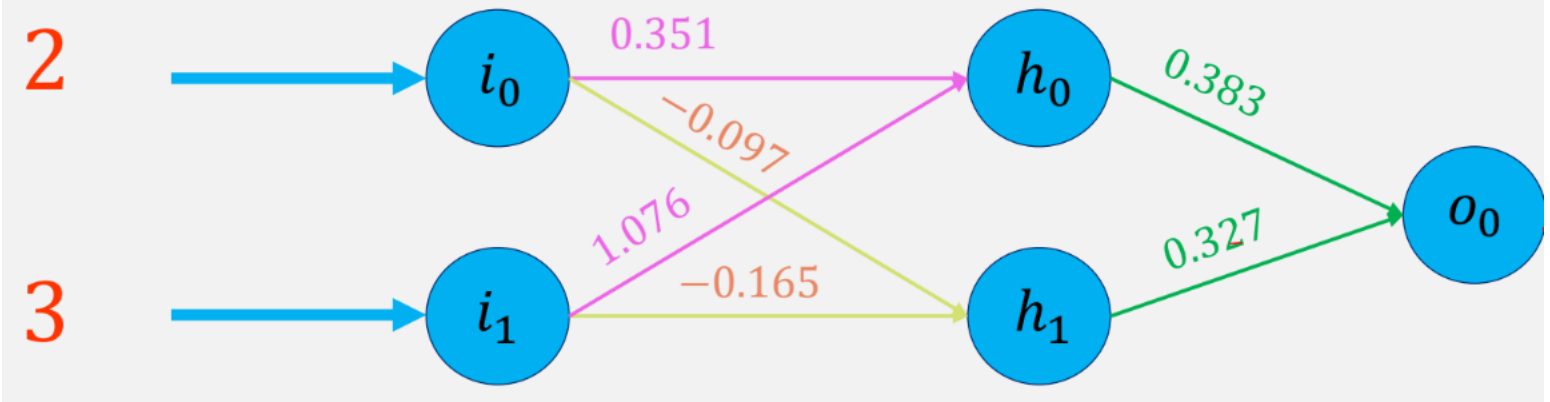
$$\frac{\partial loss}{\partial w_{1,0}^{1}} = \frac{\partial loss}{\partial Prediction} \times \frac{\partial Prediction}{\partial w_{1,0}^{1}}$$



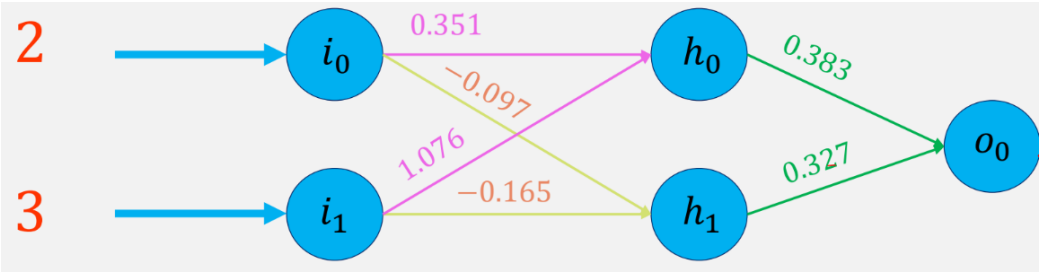
Actual y = 1

Example

$$\frac{\partial loss}{\partial w_{1,0}^{1}} = \frac{\partial loss}{\partial Prediction} \times \frac{\partial Prediction}{\partial w_{1,0}^{1}}$$



$$Prediction = (2 \times 0.351 + 3 \times 1.076)\,0.383 + (2 \times -0.097 + 3 \times -0.165)\,0.327 = 1.730$$



$$\frac{\partial loss}{\partial w^1_{1,0}} = \frac{\partial loss}{\partial Prediction} \times \frac{\partial Prediction}{\partial w^1_{1,0}}$$

Prediction = $(x_0 w^0_{0,0} + x_0 w^0_{1,0})\, w^1_{0,0} + (x_0 w^0_{0,1} + x_0 w^0_{1,1})\, w^1_{1,0}$

$$\Delta = Prediction - actual = 1.730 - 1 = 0.730$$

h_1 = -0.097*2 - 3*0.165 = -0.689

$$\frac{\partial Error}{\partial w^1_{1,0}} = \frac{\partial Error}{\partial Prediction} \times \frac{\partial Prediction}{\partial w^1_{1,0}} \quad \Rightarrow \Delta h_1$$

$$\frac{\partial Error}{\partial w^1_{1,0}} = (0.730) \times (-0.689) = \boxed{-0.502}$$

$$\frac{\partial Error}{\partial w_{1,0}^{1}} = \frac{\partial Error}{\partial Prediction} \times \frac{\partial Prediction}{\partial w_{1,0}^{1}} \quad \Rightarrow \Delta h_1$$

$$\frac{\partial Error}{\partial w_{1,0}^{1}} = (0.730) \times (-0.689) = \boxed{-0.502}$$

$$\boldsymbol{w^{+} = w^{-}} - \alpha \frac{\boldsymbol{\partial L}}{\boldsymbol{\partial w}}$$

$$w_{1,0_{\text{new}}}^{1} = 0.327 - (-0.502) = 0.829$$